Devin Moore
CIS 344 Solo Group Project
Topic: Conference Management System
Video Presentation: video_presentation
Github: conference_management_system

Conference Management System	Handle speaker submissions, attendee registrations, and session scheduling.	Joins for speakers, sessions, and attendees; constraints for room capacity; transactions for registration payments; secure user authentication.

Project Report:

Project Plan:

The plan for this project was to implement a system for scheduling conferences. These conferences would then be able to take in registrations from both attendees and speakers for the conferences. This entailed a database to hold all of the necessary information for the users and conference sessions. In order to interact with this database, I would use a php front end.

Technical Details:

The first step in the development process was designing the database. I brainstormed the initial tables and interactions between tables and how they would be joined together. I first created a Conceptual Data Model to outline all of the tables, relations, primary keys, and foreign keys.

Entities

Users (Represents all users in the system)
- UserID (PK)
- Name
- Email
- Password
- Role(speaker/attendee/admin)

Speakers (A user who submits presentations)
- SpeakerID (PK)
- UserID (FK)
- ExpertiseArea

Attendees (A user who registers for the conference)
- AttendeeID (PK)
- UserID (FK)

Conferences (Individual conference sessions)
- ConferenceID (PK)
- Name
- Description
- Datetime
- RoomID (FK)

Rooms
- RoomID (PK)
- RoomNum

SpeakerSubmission (Speakers submitting for a conference session)
- SubmissionID (PK)
- SpeakerID (FK)
- SubmissionDate
- ApprovalStatus

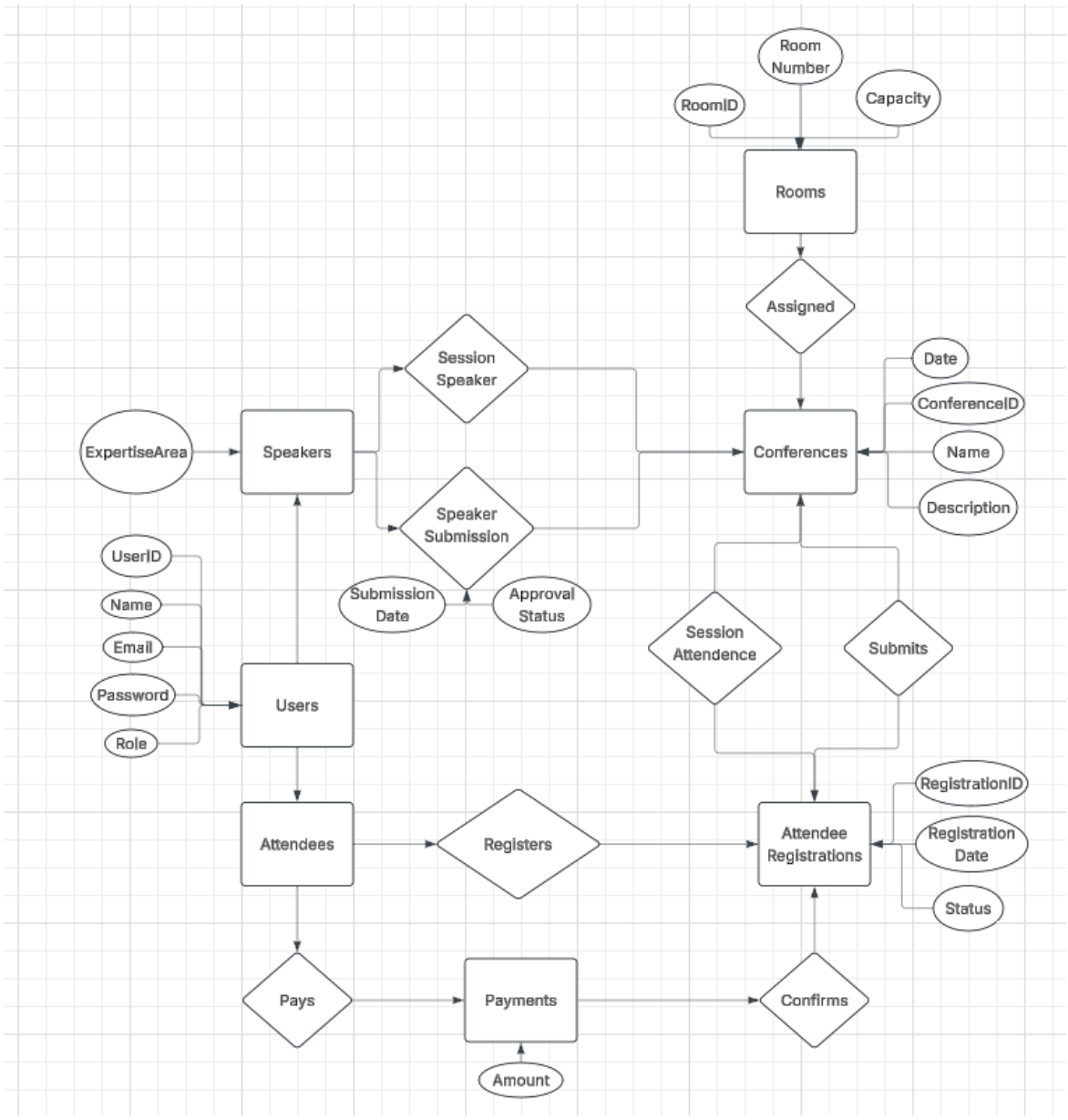
AttendeeRegistration (Conference Registrations)
- RegistrationID (PK)
- AttendeeID (FK)
- ConferenceID (FK)
- RegistrationDate
- Status

Payments
- PaymentID (PK)
- RegistrationID (FK)
- Amount

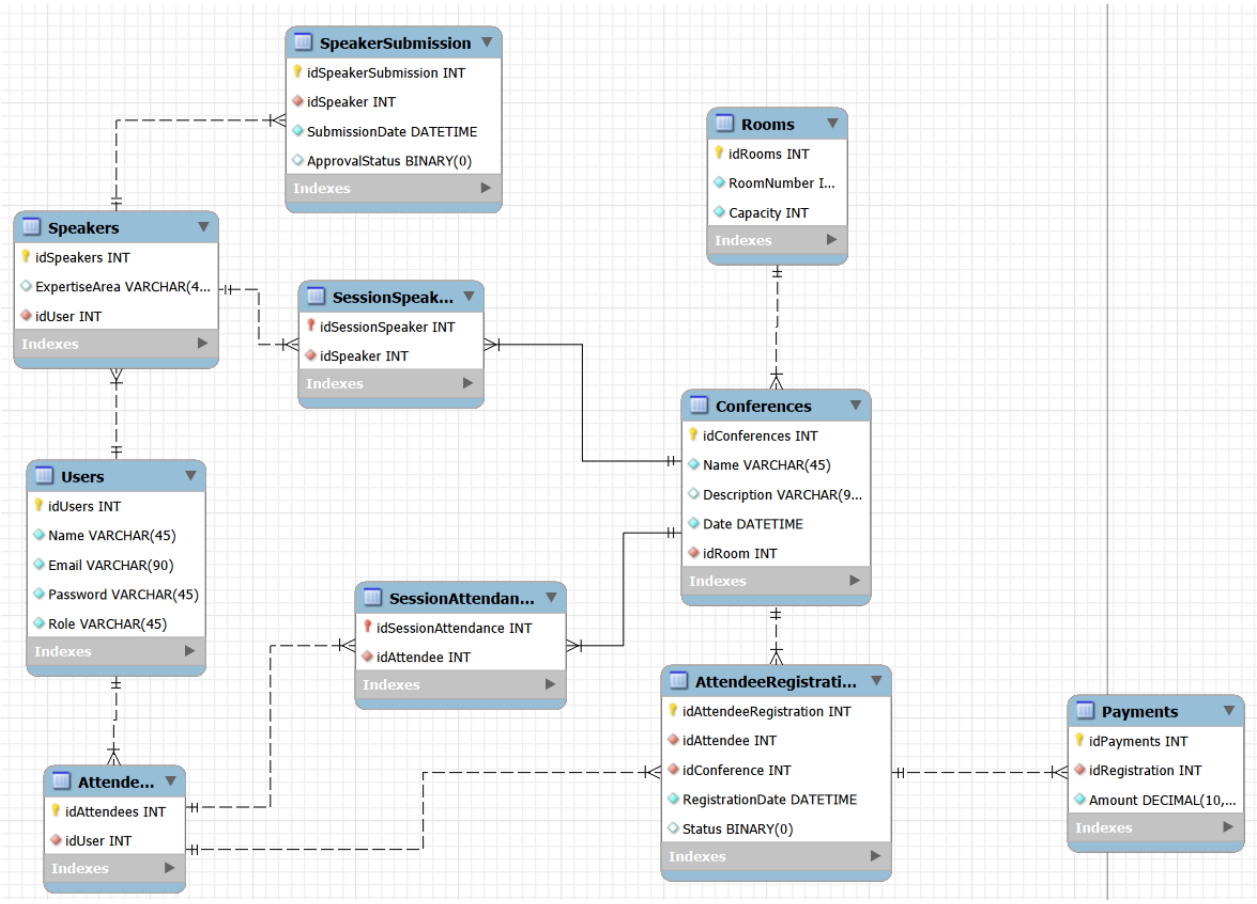
SessionSpeaker (Allows Multiple Speakers per session)
- SessionID (PK)
- SpeakerID (FK)

SessionAttendance (allows attendees to enroll in sessions)
- SessionID (PK)
- AttendeeID (FK)

From this Conceptual Data Model, I used it to create an Entity Relation Diagram to outline how these tables related to one another.

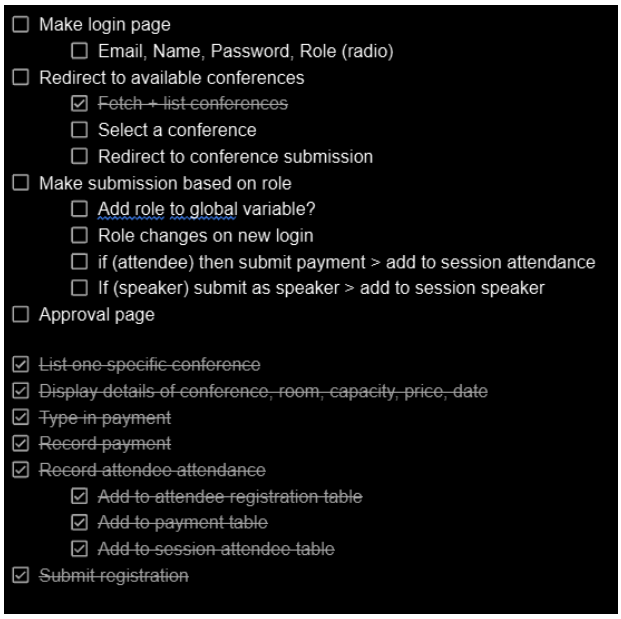


After I had the ER diagram I used the MySQL workbench to create the database and see how closely the generated ER diagram matched my built-up ER diagram.



After building the database on the MySQL workbench I had to transfer the database design to XAMPP for hosting on my local machine.

Now with the database built I began working on the front end to interact with the database. To better provide myself with direction on the steps necessary to complete all the requirements, I created “TODO’s” in the form of checklists:



These checklists were an integral part of the workflow process for the development of the Conference Management system. For the rest of the workflow, I would be mostly creating various php files to get data from the database. This was achieved with creating functions in a custom class:

```php
public function getConferences()
{
    // query for all conferences
    // $sql = "SELECT * FROM conferences ORDER BY idConferences DESC";

    // make sql query
    $sql = "SELECT
                c.idConferences,
                c.Name AS Conference_Name,
                c.Description,
                c.Date AS Conference_Date,
                c.Price,
                r.RoomNumber AS Room_Number,
                r.Capacity
            FROM conferences c
            JOIN rooms r
            ON c.idroom = r.idrooms";

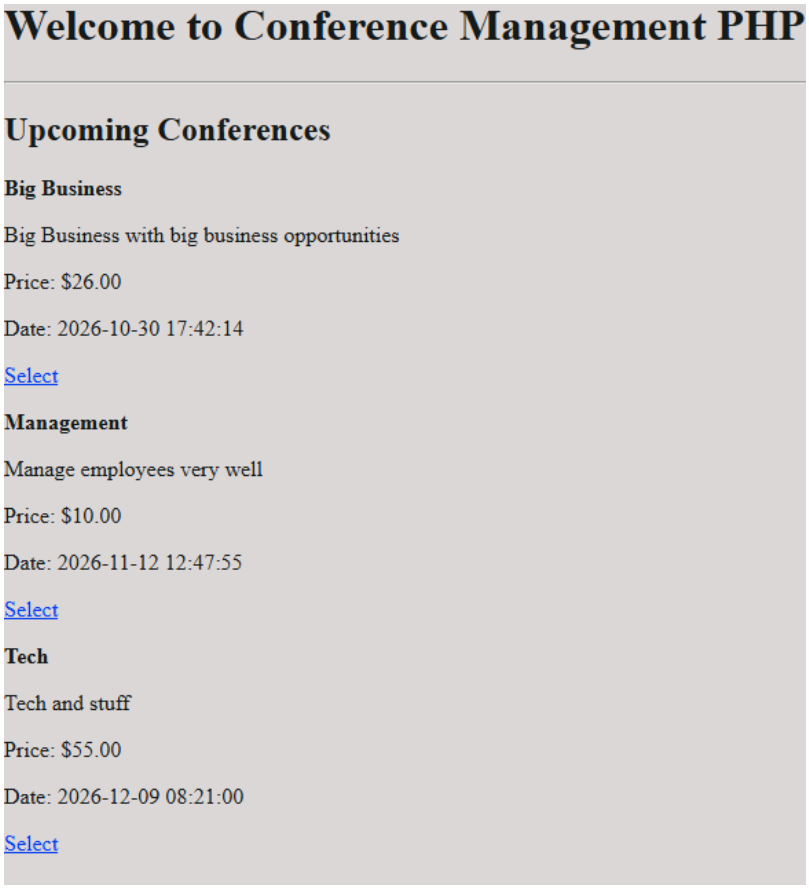
    // make query and get results
    $result = $this->conn->query($sql);
    // fetch results in an array
    return $result->fetch_all(MYSQLI_ASSOC);
}

public function getSessionSpeakers($idConference)
{
    // make sql query
    $sql = "SELECT
                c.idConferences,
                c.Name AS Conference_Name,
                c.Description,
                c.Date AS Conference_Date,
                r.RoomNumber AS Room_Number,
                r.Capacity,
                s.ExpertiseArea,
                u.Name
            FROM conferences c
            JOIN rooms r ON c.idroom = r.idrooms
            JOIN sessionspeaker ss ON c.idConferences = ss.idSessionSpeaker
            JOIN speakers s ON ss.idSpeaker = s.idSpeakers
            JOIN users u ON s.idUser = u.idUsers
            WHERE idConferences = $idConference";
```

For the website, I initially was developing a “user-friendly” interface, but due to time constraints I switched to a more “minimum viable product” (MVP) approach in order to meet deadlines. This can be seen on the index > speaker/attendee submission. Where it is easy to go from the home page, which lists all conferences, to an individual session page where a user can sign up for that specific conference. This is all achieved with functional php code.

The following code is the index.php which takes all entries from the conference table, joins with the room table, and displays the result. Each of these entries then also includes a button where you can select that specific conference to register for.

```php
<?php foreach ($conferences as $row): ?>
    <p>
        <strong>
            <?php echo htmlspecialchars($row['Conference_Name']) ?>
        </strong>
    </p>
    <p>
        <?php echo htmlspecialchars($row["Description"]) ?>
    </p>
    <p>
        Price: $<?php echo htmlspecialchars($row["Price"]) ?>
    </p>
    <p>
        Date: <?php echo htmlspecialchars($row['Conference_Date']) ?>
    </p>
    <a type='submit' href="/conferences/registration.php?id=<?php echo $row['idConferences'] ?>" name='selected'>Select</a>
<?php endforeach; ?>
```



Welcome to Conference Management PHP

Upcoming Conferences

Big Business

Big Business with big business opportunities

Price: $26.00

Date: 2026-10-30 17:42:14

Select

Management

Manage employees very well

Price: $10.00

Date: 2026-11-12 12:47:55

Select

Tech

Tech and stuff

Price: $55.00

Date: 2026-12-09 08:21:00

Select

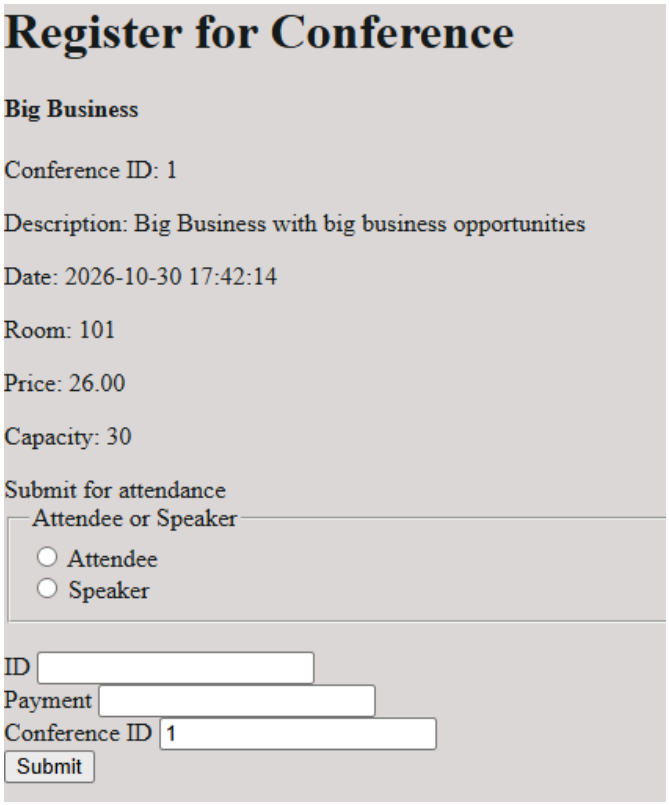
The following page then allows a user to input all of their information and inserts their registration to the database. It dynamically changes which tables to insert into based on user inputs. The page also lists additional information about the conference such as room number and capacity.

```php
<?php if ($conference): ?>
    <h4><?php echo htmlspecialchars($conference['Conference_Name']); ?></h4>
    <p>Conference ID: <?php echo htmlspecialchars($conference['idConferences']); ?></p>
    <p>Description: <?php echo htmlspecialchars($conference['Description']); ?></p>
    <p>Date: <?php echo date($conference['Conference_Date']); ?></p>
    <p>Room: <?php echo htmlspecialchars($conference['Room_Number']); ?></p>
    <p>Price: <?php echo htmlspecialchars($conference['Price']); ?></p>
    <p>Capacity: <?php echo htmlspecialchars($conference['Capacity']); ?></p>
<?php else: ?>
    <h5>No such conference exists!</h5>
<?php endif; ?>

<form method="POST" action="SubmissionCheck.php">
    <label for="submittion">Submit for attendance</label>
    <br>

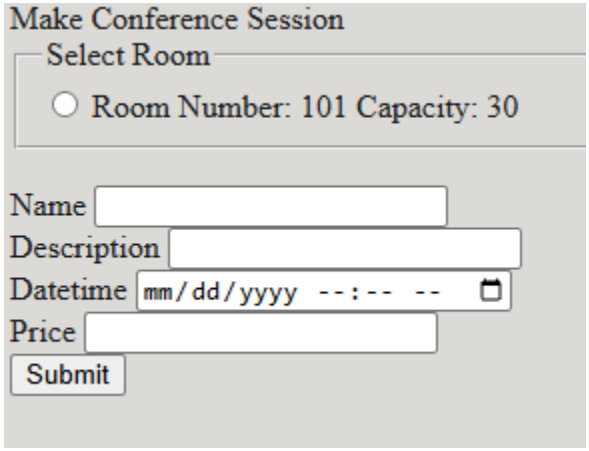
    <fieldset>
        <Legend>Attendee or Speaker</Legend>
        <div>
            <input type="radio" name="role" value="Attendee" id="Attendee">
            <label for="Attendee">Attendee</label><br>
            <input type="radio" name="role" value="Speaker" id="Speaker">
            <label for="Speaker">Speaker</label><br>
        </div>
    </fieldset>

    <br>
    <label for="id">ID</label>
    <input name="id" type="number">
    <br>
    <label for="payment">Payment</label>
    <input name="payment" type="number">
    <br>
    <label for="idConference">Conference ID</label>
    <input name="idConference" type="number" value="<?php echo htmlspecialchars($conference['idConferences']) ?>">
    <br>
    <input type="submit" value="Submit">
</form>
```



At this point I felt like I was starting to get close on time with many more things to implement so to interact with the other forms of the database, you would need to know the files in order to go to the correct url locations.

This is first seen in the conference form. Which dynamically lists all of the rooms in the database for selection when creating a conference session. Currently only one is listed because there is only one in the database but the code allows for multiple to be listed and selected via a radio element.



The handling of this form is done via php code.

```php
if (empty($_POST['room']) && empty($_POST['name'])) {
    echo "Did not fill out all formed needed  for submission!";
} else {
    // protecting data going into database
    $conferenceName = mysqli_real_escape_string($conn, $_POST['name']);
    $description = mysqli_real_escape_string($conn, $_POST['description']);
    $datetime = mysqli_real_escape_string($conn, $_POST['datetime']);
    $room = mysqli_real_escape_string($conn, $_POST['room']);
    $price = mysqli_real_escape_string($conn, $_POST['price']);

    $sql = "INSERT INTO conferences(Name, Description, Date, idRoom, Price)
            VALUES ('$conferenceName', '$description', '$datetime', '$room', '$price');";

    // SAVE TO DB AND CHECK
    if (mysqli_query($conn, $sql)) {
        // success
        echo 'Successful entry';
        mysqli_close($conn);
        header("Location: index.php");
    } else {
        // failure
        echo 'query error speakersubmission: ' . mysqli_error($conn);
    }

    mysqli_close($conn);
```

Now to list all of the speakers and attendees for an individual conference session was quite the endeavor. This required many different joins on multiple tables which are all linked by various ids and foreign keys. I created two different functions to handle the speaker joins and the attendee joins. Each of these queries had approx 5 different tables for me to join.

```php
public function getSessionAttendees($idConference)
{
    // make sql query
    $sql = "SELECT
                    c.idConferences,
                    c.Name AS Conference_Name,
                    c.Description,
                    c.Date AS Conference_Date,
                    r.RoomNumber AS Room_Number,
                    r.Capacity,
                    u.Name
                FROM conferences c
                JOIN rooms r ON c.idroom = r.idrooms
                JOIN sessionattendees sa ON c.idConferences = sa.idSessionAttendee
                JOIN attendees a ON sa.idAttendee = a.idAttendees
                JOIN users u ON a.idUser = u.idUsers
                WHERE idConferences = $idConference";

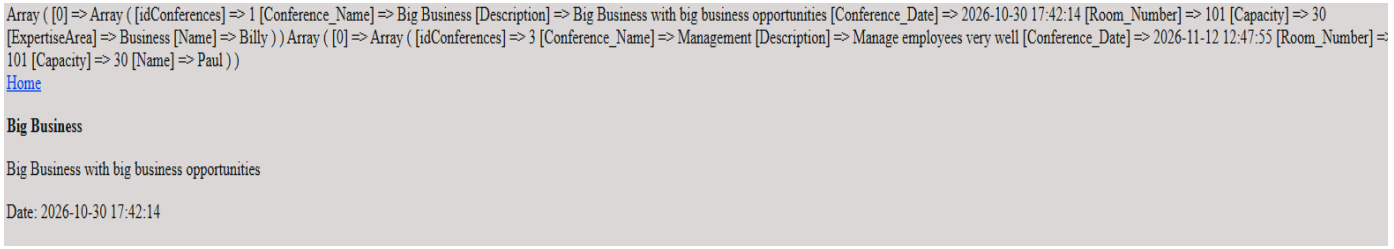
    // make query and get results
    $result = $this->conn->query($sql);
    // fetch results in an array
    return $result->fetch_all(MYSQLI_ASSOC);
}
```

In the interest of time, I did not add the functionality to dynamically list the various conferences. Instead I hard coded which conference to list in the code. I also did not create the html elements to display the results in a “user-friendly” manner. Instead, I displayed the resulting array from the query.

```php
$sessionSpeakers = $conference->getSessionSpeakers(1);
$sessionAttendees = $conference->getSessionAttendees(3);
print_r($sessionSpeakers);
print_r($sessionAttendees);
?>

<!DOCTYPE html>
<html>
<?php include('templates/header.php') ?>

<?php foreach ($sessionSpeakers as $session): ?>
    <p>
        <strong>
            <?php echo htmlspecialchars($session['Conference_Name']) ?>
        </strong>
    </p>
    <p>
        <?php echo htmlspecialchars($session["Description"]) ?>
    </p>
    <p>
        Date: <?php echo htmlspecialchars($session['Conference_Date']) ?>
    </p>
<?php endforeach; ?>
```


Array ([0] => Array ([idConferences] => 1 [Conference_Name] => Big Business [Description] => Big Business with big business opportunities [Conference_Date] => 2026-10-30 17:42:14 [Room_Number] => 101 [Capacity] => 30 [ExpertiseArea] => Business [Name] => Billy)) Array ([0] => Array ([idConferences] => 3 [Conference_Name] => Management [Description] => Manage employees very well [Conference_Date] => 2026-11-12 12:47:55 [Room_Number] => 101 [Capacity] => 30 [Name] => Paul))

Home

Big Business

Big Business with big business opportunities

Date: 2026-10-30 17:42:14

This query does list every attendee and speaker for that specific conference session. If I had more time then I would individually display everything in a dynamic manner for a better user experience.

A later addition to the database was creating a trigger that checks if the session attendees does not exceed the room capacity. I could have done this check either client side or server side. I chose to do it server side because I thought it would be more secure for the integrity of the database.

```
-- Count current attendees registered for this session
SELECT COUNT(*)
INTO current_count
FROM sessionattendees
WHERE idSessionAttendee = NEW.idSessionAttendee;

-- Get the room capacity
SELECT r.capacity
INTO room_capacity
FROM conferences c
JOIN rooms r ON c.idroom = r.idrooms
WHERE c.idConferences = NEW.idSessionAttendee;
```

```
-- Prevent registration if room is full
IF current_count >= room_capacity THEN
    SIGNAL SQLSTATE '45000'
    SET MESSAGE_TEXT = 'Room capacity exceeded';
END IF;
```

The final task I worked on would be a login form. This login form would need secure authentication. I did this with the built-in php password hashing function. I ran out of time to implement database functionality but that would be the next step to complete the login flow.

```
<a href="HomePage.php">To Home Page</a>
<form action='LoginPage.php' method='POST'>
    username:<br>
    <input type="text" name="username"><br>
    password:<br>
    <input type="password" name="password"><br>
    <input type="submit" name="login" value="login">

</form>

</body>

</html>

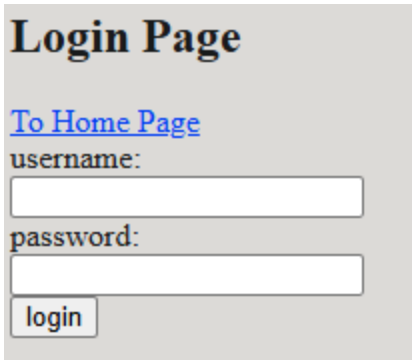
<?php

if (isset($_POST['login'])) {

    if (!empty($_POST['username']) && !empty($_POST['password'])) {
        $_SESSION["username"] = $_POST["username"];
        $_SESSION["password"] = password_hash($_POST["password"], PASSWORD_DEFAULT);

        echo $_SESSION["username"] . "<br>";
        echo $_SESSION["password"] . "<br>";

        header("Location: HomePage.php");
    } else {
        echo "Missing password/username. <br>";
    }
}
```



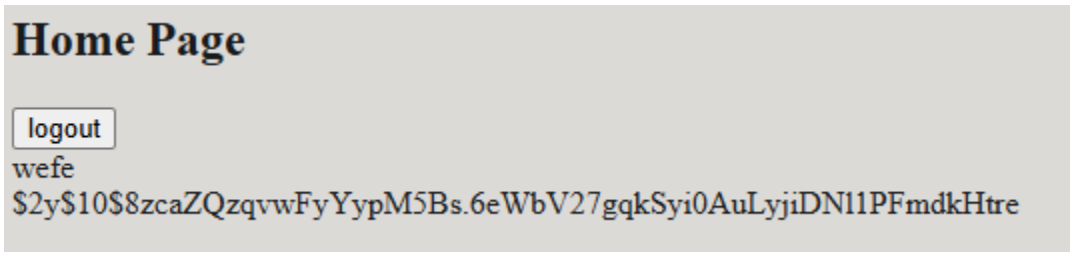

```php
<form action="HomePage.php" method="POST">
    <input type="submit" name="logout" value="logout">
</form>

</body>

</html>

<?php
echo $_SESSION["username"] . "<br>";
echo $_SESSION["password"] . "<br>";

if (isset($_POST["logout"])) {
    session_destroy();
    header("Location: LoginPage.php");
}
```



USER GUIDE:

- LoginPage.php > login page
- index.php > Main page that lists conference sessions
- registration.php?id="idConference" > registration for both speakers and attendees.
- conferenceform.php > form to submit new conference
- conferencesession.php > display all speakers and attendees for an individual conference session. (not completely functional and currently hard coded specific session)

To allow xampp to run mysql, search for services > Stop MySQL80 from running (This process is used by the MySQL application)